

Import libraries

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    confusion_matrix,
    ConfusionMatrixDisplay,
    classification_report,
    accuracy_score,
    precision_score,
    recall_score
)
```

Import data

```
In [14]: # import the dataset
df = pd.read_csv(r"C:\Users\MOSHOOD\Desktop\3MTT\LRA\Invistico_Airline.csv")

df.head()
```

```
Out[14]:
```

	satisfaction	Customer Type	Age	Type of Travel	Class	Flight Distance	Seat comfort	Departure/Arrival time convenient	F d
0	satisfied	Loyal Customer	65	Personal Travel	Eco	265	0	0	
1	satisfied	Loyal Customer	47	Personal Travel	Business	2464	0	0	
2	satisfied	Loyal Customer	15	Personal Travel	Eco	2138	0	0	
3	satisfied	Loyal Customer	60	Personal Travel	Eco	623	0	0	
4	satisfied	Loyal Customer	70	Personal Travel	Eco	354	0	0	

5 rows × 22 columns



Data Exploration

```
In [17]: df.info()

df.describe()

df.isnull().sum()

df['satisfaction'].value_counts()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 129880 entries, 0 to 129879
Data columns (total 22 columns):
#   Column                                     Non-Null Count  Dtype
---  -
0   satisfaction                             129880 non-null  str
1   Customer Type                           129880 non-null  str
2   Age                                      129880 non-null  int64
3   Type of Travel                          129880 non-null  str
4   Class                                    129880 non-null  str
5   Flight Distance                         129880 non-null  int64
6   Seat comfort                            129880 non-null  int64
7   Departure/Arrival time convenient       129880 non-null  int64
8   Food and drink                          129880 non-null  int64
9   Gate location                           129880 non-null  int64
10  Inflight wifi service                   129880 non-null  int64
11  Inflight entertainment                  129880 non-null  int64
12  Online support                          129880 non-null  int64
13  Ease of Online booking                  129880 non-null  int64
14  On-board service                       129880 non-null  int64
15  Leg room service                       129880 non-null  int64
16  Baggage handling                       129880 non-null  int64
17  Checkin service                        129880 non-null  int64
18  Cleanliness                            129880 non-null  int64
19  Online boarding                        129880 non-null  int64
20  Departure Delay in Minutes              129880 non-null  int64
21  Arrival Delay in Minutes                129487 non-null  float64
dtypes: float64(1), int64(17), str(4)
memory usage: 21.8 MB
```

```
Out[17]: satisfaction
satisfied      71087
dissatisfied   58793
Name: count, dtype: int64
```

Data Cleaning

```
In [5]: df.isnull().sum()
```

```
Out[5]: satisfaction      0
Customer Type            0
Age                      0
Type of Travel           0
Class                    0
Flight Distance          0
Seat comfort              0
Departure/Arrival time convenient  0
Food and drink           0
Gate location            0
Inflight wifi service    0
Inflight entertainment   0
Online support           0
Ease of Online booking   0
On-board service         0
Leg room service         0
Baggage handling         0
Checkin service          0
Cleanliness              0
Online boarding          0
Departure Delay in Minutes  0
Arrival Delay in Minutes  393
dtype: int64
```

```
In [19]: # Fill missing values in 'Arrival Delay in Minutes' column with the median value
df['Arrival Delay in Minutes'] = df['Arrival Delay in Minutes'].fillna(
    df['Arrival Delay in Minutes'].median()
)
```

The arrival delay time contains 393 missing values, this was handle by fill them with median value of the overrrall datasets

Inspect Target Variable

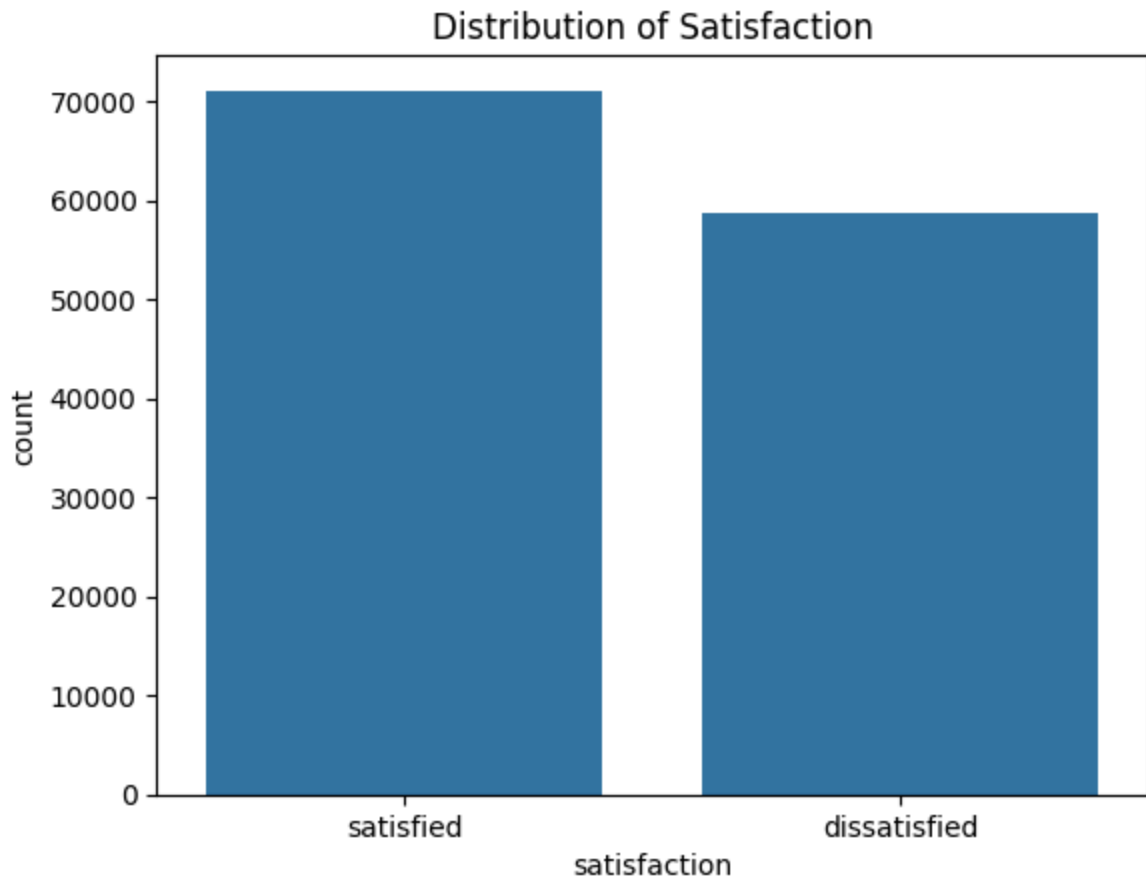
```
In [22]: # Check for missing values again
df.isnull().sum()
```

```
Out[22]: satisfaction      0
         Customer Type    0
         Age              0
         Type of Travel    0
         Class            0
         Flight Distance   0
         Seat comfort      0
         Departure/Arrival time convenient 0
         Food and drink    0
         Gate location     0
         Inflight wifi service 0
         Inflight entertainment 0
         Online support     0
         Ease of Online booking 0
         On-board service  0
         Leg room service  0
         Baggage handling  0
         Checkin service   0
         Cleanliness       0
         Online boarding   0
         Departure Delay in Minutes 0
         Arrival Delay in Minutes 0
         dtype: int64
```

```
In [24]: # check the distribution of the target variable
         df['satisfaction'].value_counts()
```

```
Out[24]: satisfaction
         satisfied      71087
         dissatisfied   58793
         Name: count, dtype: int64
```

```
In [ ]: # visualize the distribution of the target variable
         sns.countplot(x='satisfaction', data=df)
         plt.title("Distribution of Satisfaction")
         plt.show()
```



The target column used to classify the customer satisfaction contain 71087 customers was reported satisfy with airline service while 58793 did not satisfy with the service of the airline service.

Encode the Variables

```
In [26]: # Select columns with object data type
df.select_dtypes(include='object').columns
```

C:\Users\MOSHOOOD\AppData\Local\Temp\ipykernel_11480\2340838403.py:2: Pandas4Warning: For backward compatibility, 'str' dtypes are included by select_dtypes when 'object' dtype is specified. This behavior is deprecated and will be removed in a future version. Explicitly pass 'str' to `include` to select them, or to `exclude` to remove them and silence this warning.

See https://pandas.pydata.org/docs/user_guide/migration-3-strings.html#string-migration-select-dtypes for details on how to write code that works with pandas 2 and 3.

```
df.select_dtypes(include='object').columns
```

```
Out[26]: Index(['satisfaction', 'Customer Type', 'Type of Travel', 'Class'], dtype='str')
```

```
In [27]: encoder = LabelEncoder()

categorical_cols = df.select_dtypes(include='object').columns
```

```
for col in categorical_cols:
    df[col] = encoder.fit_transform(df[col])
```

C:\Users\MOSHOOOD\AppData\Local\Temp\ipykernel_11480\1448182830.py:3: Pandas4Warning: For backward compatibility, 'str' dtypes are included by select_dtypes when 'object' dtype is specified. This behavior is deprecated and will be removed in a future version. Explicitly pass 'str' to `include` to select them, or to `exclude` to remove them and silence this warning.

See https://pandas.pydata.org/docs/user_guide/migration-3-strings.html#string-migration-select-dtypes for details on how to write code that works with pandas 2 and 3.

```
categorical_cols = df.select_dtypes(include='object').columns
```

Feature Selection

```
In [29]: X = df.drop('satisfaction', axis=1)

y = df['satisfaction']
```

Train-Test Split

```
In [30]: X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

Logistic Regression Model

```
In [31]: # Split the data
X = df.drop('satisfaction', axis=1)
y = df['satisfaction']

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

# Scale the features
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Train the model
```

```
model = LogisticRegression(max_iter=5000)

model.fit(X_train, y_train)
```

Out[31]:

▼ **LogisticRegression** ⓘ ?

- Parameters
- Fitted attributes

The Logistic Regression model was successfully trained using the standardized training dataset. Before fitting the model, the data was divided into training (80%) and testing (20%) sets to ensure that model performance could be evaluated on unseen data. Feature scaling was performed using `StandardScaler` to place all predictor variables on a comparable scale, which improves the convergence and stability of the Logistic Regression algorithm. The model converged successfully within the specified maximum number of iterations (`max_iter=5000`), indicating that the optimization process found a stable set of coefficients for predicting passenger satisfaction. At this stage, the model has learned the relationship between the predictor variables and the target variable (`satisfaction`) and is ready to make predictions on the test dataset.

Predictions

```
In [32]: y_pred = model.predict(X_test)
```

```
In [33]: y_prob = model.predict_proba(X_test)

y_prob[:10]
```

```
Out[33]: array([[0.07525387, 0.92474613],
                [0.93300399, 0.06699601],
                [0.16941293, 0.83058707],
                [0.20956885, 0.79043115],
                [0.79965525, 0.20034475],
                [0.19009253, 0.80990747],
                [0.96697993, 0.03302007],
                [0.32825807, 0.67174193],
                [0.04468297, 0.95531703],
                [0.66289111, 0.33710889]])
```

The output above displays the predicted probabilities of each passenger belonging to the two satisfaction classes. Each row corresponds to a passenger in the test dataset, where the first value represents the probability of the passenger being Dissatisfied (Class 0) and the second value represents the probability of the passenger being Satisfied (Class 1). For example, the first passenger has a 7.53% probability of being Dissatisfied and a 92.47% probability of being Satisfied. Since the probability of being Satisfied is greater than 50%, the

model classifies this passenger as Satisfied. Similarly, the second passenger has a 93.30% probability of being Dissatisfied and only a 6.70% probability of being Satisfied, so the model predicts this passenger as Dissatisfied. These probability estimates provide more information than the predicted class labels alone, as they indicate the model's level of confidence in each prediction.

```
In [35]: # Evaluate the accuracy of the model
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)
```

Accuracy: 0.8257622420696027

The Logistic Regression model achieved an **accuracy of 82.58%**, indicating that it correctly classified approximately **83 out of every 100 passengers** in the test dataset. This suggests that the model has a good overall ability to distinguish between satisfied and neutral/dissatisfied passengers. While the accuracy indicates strong predictive performance, it should not be used as the sole measure of model effectiveness. In classification problems, accuracy alone may not fully capture the model's performance, particularly if the classes are imbalanced. Therefore, additional evaluation metrics such as **Precision, Recall, F1-score**, and the **Confusion Matrix** are also examined to provide a more comprehensive assessment of the model's ability to correctly identify satisfied and dissatisfied passengers.

```
In [36]: # Evaluate the recall of the model
recall = recall_score(y_test, y_pred)

print("Recall:", recall)
```

Recall: 0.8408502901894972

The Logistic Regression model achieved a **recall of 84.09%**, meaning that it correctly identified approximately **84 out of every 100 passengers who were actually satisfied**. In other words, the model successfully detected the majority of satisfied passengers while missing only a relatively small proportion. A high recall is important in this context because it indicates that the model is effective at recognizing satisfied customers. This enables the airline to better understand the characteristics associated with customer satisfaction and to accurately identify passengers who are likely to have had a positive travel experience. Although the recall value is high, it should be interpreted alongside **precision** to ensure that the model not only identifies most satisfied passengers but also minimizes incorrect positive predictions. Together with the Confusion Matrix and other evaluation metrics, recall provides a more comprehensive assessment of the model's classification performance.

```
In [37]: print(classification_report(y_test, y_pred))
```


	precision	recall	f1-score	support
0	0.81	0.81	0.81	11675
1	0.84	0.84	0.84	14301
accuracy			0.83	25976
macro avg	0.82	0.82	0.82	25976
weighted avg	0.83	0.83	0.83	25976

The classification report provides a comprehensive evaluation of the Logistic Regression model by summarizing its **precision**, **recall**, **F1-score**, and the number of observations (**support**) for each class. For **Class 0 (Neutral or Dissatisfied)**, the model achieved a **precision of 81%**, indicating that when it predicted a passenger as neutral or dissatisfied, the prediction was correct 81% of the time. It also achieved a **recall of 81%**, meaning that it correctly identified 81% of all passengers who were actually neutral or dissatisfied. The **F1-score of 81%** demonstrates a balanced performance between precision and recall for this class. For **Class 1 (Satisfied)**, the model achieved both a **precision and recall of 84%**, indicating that it reliably identifies satisfied passengers while correctly capturing most of the actual satisfied cases. The corresponding **F1-score of 84%** confirms a strong balance between prediction accuracy and the ability to detect satisfied passengers. The model achieved an overall **accuracy of 82.58%**, demonstrating good classification performance across the test dataset. Additionally, the **macro average** values of **82%** indicate that the model performs consistently across both classes without being overly influenced by the larger class. The **weighted average** values of **83%** further confirm that the model maintains strong overall performance while accounting for the class distribution. Overall, the classification report suggests that the Logistic Regression model is effective at distinguishing between satisfied and neutral/dissatisfied passengers, with slightly stronger performance in identifying satisfied passengers. These results indicate that the model is reliable for supporting customer satisfaction analysis and can provide valuable insights for improving airline service quality and customer retention strategies.

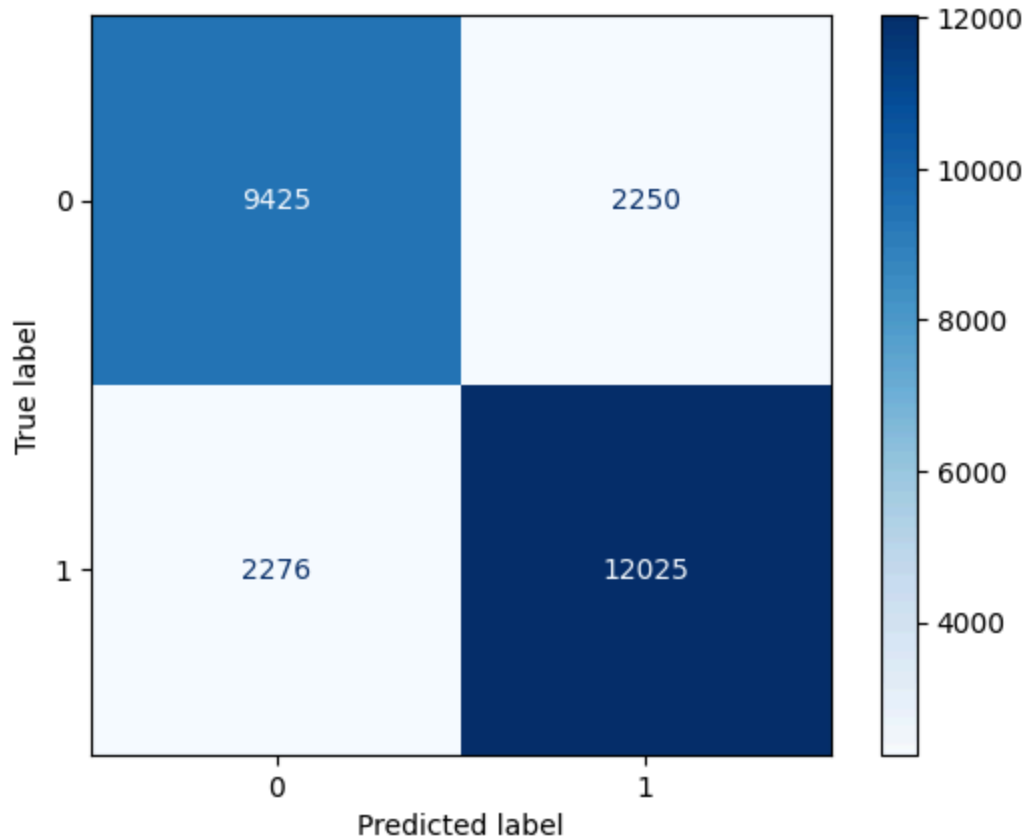
Confusion Matrix

```
In [38]: cm = confusion_matrix(y_test, y_pred)

disp = ConfusionMatrixDisplay(confusion_matrix=cm)

disp.plot(cmap='Blues')

plt.show()
```



The model correctly classified 9,425 passengers as Dissatisfied (True Negatives) and 12,025 passengers as Satisfied (True Positives). These correct classifications account for the majority of the test observations, demonstrating that the model effectively distinguishes between the two satisfaction classes. However, the model also made some misclassifications. It incorrectly predicted 2,250 passengers as Satisfied when they were actually Neutral or Dissatisfied (False Positives). In addition, 2,276 passengers who were actually Satisfied were incorrectly classified as Neutral or Dissatisfied (False Negatives). Overall, the confusion matrix indicates that the model performs well, correctly classifying 21,450 out of 25,976 passengers, which corresponds to an overall accuracy of approximately 82.58%. The relatively balanced number of false positives and false negatives suggests that the model does not exhibit a strong bias toward either class. This balanced performance is consistent with the model's precision, recall, and F1-score, indicating that it provides reliable predictions for identifying both satisfied and neutral/dissatisfied passengers.

Interpret Coefficients

```
In [39]: coefficients = pd.DataFrame({
    'Feature': X.columns,
    'Coefficient': model.coef_[0]
})

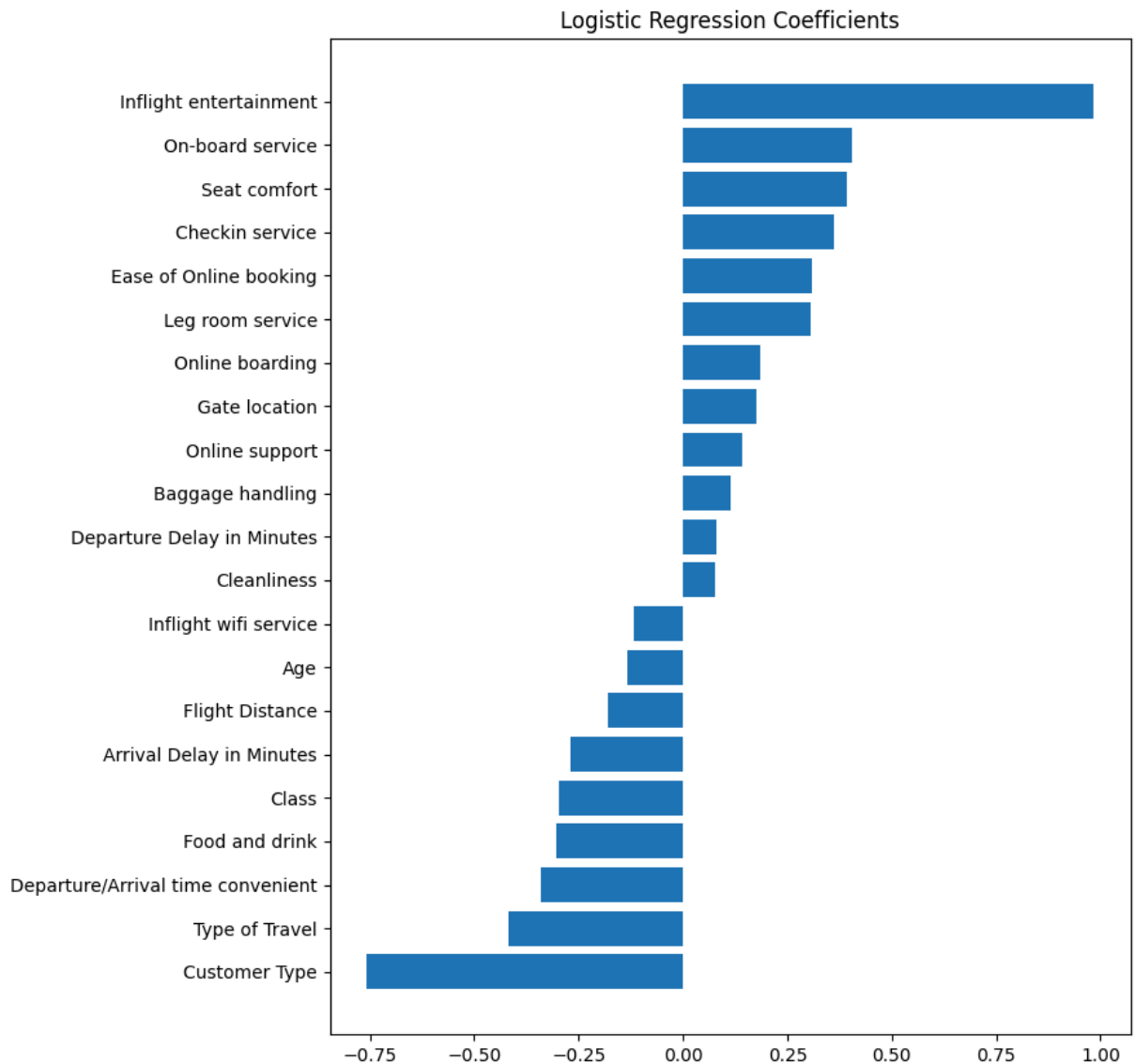
coefficients.sort_values(
    by='Coefficient',
```

```
ascending=False
)
```

Out[39]:

	Feature	Coefficient
10	Inflight entertainment	0.984318
13	On-board service	0.405525
5	Seat comfort	0.391341
16	Checkin service	0.360084
12	Ease of Online booking	0.310413
14	Leg room service	0.305734
18	Online boarding	0.185504
8	Gate location	0.174666
11	Online support	0.142962
15	Baggage handling	0.114286
19	Departure Delay in Minutes	0.079608
17	Cleanliness	0.075713
9	Inflight wifi service	-0.116871
1	Age	-0.133186
4	Flight Distance	-0.180686
20	Arrival Delay in Minutes	-0.269010
3	Class	-0.298191
7	Food and drink	-0.303581
6	Departure/Arrival time convenient	-0.340923
2	Type of Travel	-0.419212
0	Customer Type	-0.758707

```
In [40]: coefficients = coefficients.sort_values(by='Coefficient')
plt.figure(figsize=(8,10))
plt.barh(coefficients['Feature'], coefficients['Coefficient'])
plt.title("Logistic Regression Coefficients")
plt.show()
```



Interpretation

The table above presents the estimated coefficients from the Logistic Regression model, showing the direction and relative strength of each feature's association with passenger satisfaction. Positive coefficients indicate that higher values of a feature increase the likelihood that a passenger will be classified as **Satisfied**, while negative coefficients indicate a reduced likelihood of satisfaction.

Among all the predictors, **Inflight Entertainment** (0.984) is the strongest positive contributor to passenger satisfaction, suggesting that improvements in entertainment services are strongly associated with higher satisfaction levels. Other important positive factors include **On-board Service**, **Seat Comfort**, **Check-in Service**, **Ease of Online Booking**, and **Leg Room Service**, highlighting the importance of delivering a high-quality onboard and pre-flight experience.

On the other hand, **Customer Type** (-0.759) has the strongest negative coefficient. Based on the label encoding (Loyal Customer = 0, Disloyal Customer = 1), this indicates that **disloyal**

customers are less likely to be satisfied than loyal customers. Similarly, **Type of Travel** (-0.419) suggests that **personal travelers are less likely to be satisfied than business travelers**, while the negative coefficient for **Class** (-0.298) indicates that passengers traveling in **Economy or Eco Plus** are generally less likely to report satisfaction than those traveling in **Business Class**.

Additionally, **Arrival Delay in Minutes** (-0.269) demonstrates that longer arrival delays reduce the likelihood of passenger satisfaction, emphasizing the importance of operational efficiency and on-time performance.

Overall, the model indicates that passenger satisfaction is primarily driven by the quality of onboard services, comfort, and customer loyalty. These findings provide valuable insights that can help airline management prioritize investments in service quality, customer retention programs, and operational improvements to enhance the overall passenger experience.

Overall Business Interpretation

The Logistic Regression model achieved an **accuracy of 82.58%**, demonstrating good predictive performance in classifying passenger satisfaction. The analysis identified **Inflight Entertainment, On-board Service, Seat Comfort, Check-in Service, and Ease of Online Booking** as the strongest positive drivers of satisfaction. Conversely, **Disloyal Customer, Personal Travel, Economy Class, and Arrival Delay in Minutes** were associated with a lower likelihood of passenger satisfaction. These findings indicate that both service quality and operational efficiency play a critical role in shaping the overall passenger experience.

Business Recommendations

- Invest in **in-flight entertainment, seat comfort, and onboard service**, as these are the strongest drivers of passenger satisfaction.
- Enhance the **digital customer experience**, particularly online booking and check-in processes, to improve convenience.
- Reduce **arrival delays** through better operational planning and scheduling to minimize passenger dissatisfaction.
- Strengthen **customer loyalty programs** to improve retention and increase satisfaction among frequent travelers.
- Develop targeted strategies for **personal and economy-class travelers**, who are less likely to report satisfaction, by improving service quality and offering value-added benefits.

Model Limitations

The Logistic Regression model assumes a linear relationship between the predictor variables and the log-odds of the target variable, which may not capture more complex patterns in passenger satisfaction. Label encoding was used for categorical variables, which may introduce an artificial ordering for nominal categories such as travel class. Additionally, the model does not account for potential interactions between features. Future work could explore advanced classification algorithms such as Random Forest, Gradient Boosting, or XGBoost, along with one-hot encoding and hyperparameter tuning, to improve predictive performance.

Conclusion

This project successfully developed and evaluated a Logistic Regression model for predicting airline passenger satisfaction. The model demonstrated strong performance across multiple evaluation metrics, including **82.58% accuracy**, **84.09% recall**, and balanced precision and F1-scores. The results show that service quality, customer loyalty, and operational performance are the primary factors influencing passenger satisfaction. These insights can support data-driven decisions that enhance customer experience, improve retention, and strengthen the airline's competitive advantage.